HB1000 Bingo City: Technical Architecture Research Report

Document Classification: Strategic Technical Architecture
Version: 1.0
Date: March 18, 2026
Author: Learning Loop Dashboard Agent (Manus AI)
For: Tim Latimer, SIC HB1000 Solve Team

Table of Contents

1. Executive Summary
2. Master Data Architecture Overview
3. Connection 1: Source of Truth Database
4. Connection 2: Local North Star Database
5. Connection 3: Bingo Card State Engine
6. Connection 4: AI Agent Registry & Orchestration
7. Connection 5: KPI Ticker Tape Feed
8. Connection 6: Learning Loop V13 Engine
9. Connection 7: Cloud Butterfly & Wisdom Giant Capture System
10. Connection 8: Notification & Escalation Engine
11. Connection 9: Accounting & Financial Integration
12. Connection 10: Authentication, Authorization & Audit Trail
13. Recommended Tech Stack
14. Multi-Tenant Architecture Design
15. Event-Driven Architecture & Real-Time Sync
16. Phased Build Roadmap
17. Complexity & Effort Estimates

18. References

---

1. Executive Summary

This report presents the complete technical architecture required to make HB1000 Bingo City a functional, production-grade accountability dashboard. Bingo City is not a decorative visualization; it is a **project management command center** where each Pope (human leader) manages their initiative portfolio through a Bingo Card metaphor, supported by a human HB1000 team and an AI Rooftop Society.

The architecture must support 30 to 40 Bingo Buildings, each representing an initiative with its own Pope, team, Bingo Card, KPI feed, and AI agent roster. The system must handle real-time data synchronization across all buildings, event-driven accountability triggers (squares lighting up, escalation alerts, ticker tape updates), and integration with external financial systems, CRMs, communication platforms, and AI/LLM services.

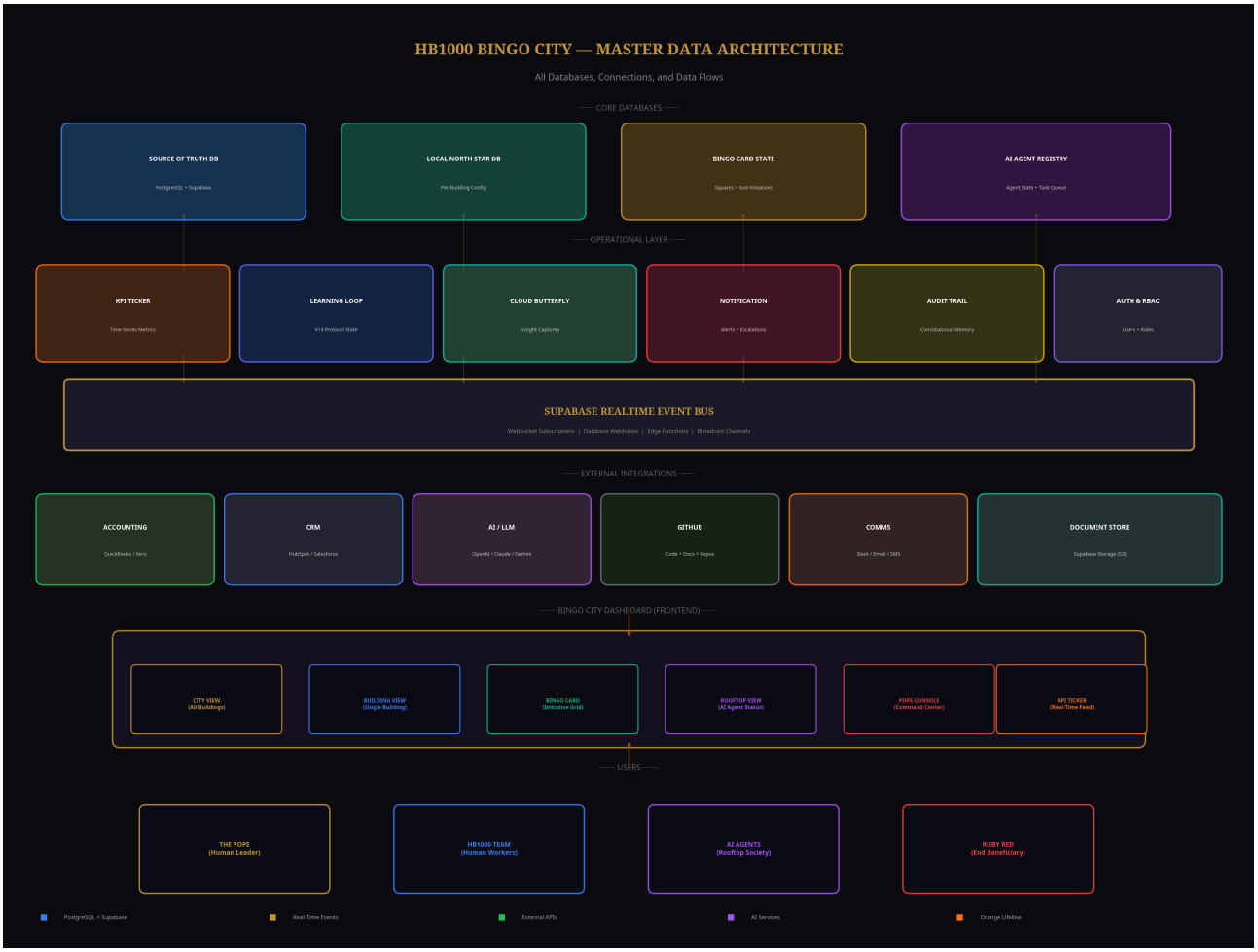
After reviewing the full HB1000 ecosystem documentation, researching multi-tenant SaaS architecture patterns, real-time synchronization strategies, and event-driven system design, the recommendation is a **Supabase-centered architecture** with PostgreSQL as the core database, Row-Level Security for multi-tenancy, Supabase Realtime for WebSocket-based live updates, and Edge Functions for serverless business logic. This stack provides the right balance of power, developer experience, cost efficiency, and scalability for the HB1000 scale of 30-40 buildings.

The total estimated build effort is **20 weeks (5 months)** across 5 phases, requiring a team of 2 full-stack developers, 1 AI/ML engineer, and 1 DevOps engineer. The phased approach allows the Pope to begin using the system after Phase 1 (4 weeks) with basic building and card management, with progressive capability additions through each subsequent phase.

---

2. Master Data Architecture Overview

The following diagram shows the complete data architecture for Bingo City, organized in six layers from core databases at the top to end users at the bottom, with the Supabase Realtime Event Bus as the central nervous system connecting everything.



The architecture is organized into six distinct layers, each serving a specific function in the accountability engine.

Layer 1 — Core Databases contains the four foundational data stores: the Source of Truth Database (canonical facts, research, citations), the Local North Star Database (per-building configuration and goals), the Bingo Card State Engine (squares, sub-initiatives, completion status), and the AI Agent Registry (agent definitions, task queues, autonomy scores). These are the permanent records of the system.

Layer 2 — Operational Databases contains the six working data stores that power daily operations: KPI Ticker (time-series financial metrics), Learning Loop (V13 protocol state and scores), Cloud Butterfly (insight captures), Notification (alerts and escalation queues), Audit Trail (constitutional memory and change logs), and Auth/RBAC (user identities and role assignments).

Layer 3 — Supabase Realtime Event Bus is the central nervous system. Every database change in Layers 1 and 2 generates a real-time event that flows through WebSocket subscriptions, database webhooks, Edge Functions, and broadcast channels. This is what makes the dashboard “live” — when a square lights up in one building, every connected dashboard sees it instantly.

Layer 4 — External Integrations connects the system to the outside world: accounting systems (QuickBooks, Xero), CRMs (HubSpot, Salesforce), AI/LLM services (OpenAI, Claude, Gemini), GitHub (code and documentation repositories), communication platforms (Slack, Email, SMS), and document storage (Supabase Storage backed by S3).

Layer 5 — Bingo City Dashboard is the frontend application with six primary views: City View (all buildings at a glance), Building View (single building deep-dive), Bingo Card (initiative grid with clickable squares), Rooftop View (AI agent status and activity), Pope Console (command center with action buttons), and KPI Ticker (real-time financial feed).

Layer 6 — Users represents the four actor types who interact with the system: the Pope (human leader, full authority), the HB1000 Team (human workers, task execution), AI Agents (Rooftop Society, automated assistance), and Ruby Red (the end beneficiary whose outcomes the entire system serves).

---

3. Connection 1: Source of Truth Database

What It Is

The Source of Truth Database is the canonical knowledge repository for the entire HB1000 ecosystem. It stores verified facts, research findings, citations, policy documents, constitutional directives, and any information that has been validated by the Watchman and stamped by the Citation Officer. Nothing in Bingo City should contradict what is in this database. It is the Fireproof Safe of the digital world — the place where truth is preserved.

What Data It Stores

Table	Purpose	Key Fields	Write Frequency
`verified_facts`	Canonical facts validated by Watchman	`id`, `building_id`, `category`, `statement`, `source_url`, `citation_id`, `verified_by`, `verified_at`, `confidence_score`	Daily
`policy_documents`	Constitutional directives, covenants	`id`, `title`, `content_md`, `version`, `approved_by`, `effective_date`, `supersedes_id`	Weekly
`research_entries`	Research findings from Owl + external	`id`, `building_id`, `topic`, `findings_md`, `sources`, `researcher_agent`, `quality_score`	Daily
`citation_registry`	Every citation ever issued	`id`, `fact_id`, `source_type`, `source_url`, `source_title`, `retrieved_at`, `still_valid`	Continuous
`north_star_global`	The master North Star (above the line)	`id`, `statement`, `bhag`, `values`, `vision`, `approved_by`, `version`	Quarterly

How It Connects to Each Building

Every Bingo Building has a `building_id` foreign key that links to the Source of Truth. When an AI agent (particularly the Owl or Citation Officer) needs to verify a claim, they query this database. The Watchman agent runs continuous validation checks against external sources to ensure facts remain current. If a fact becomes stale or contradicted, the Watchman flags it and the Citation Officer issues a correction notice that propagates to all buildings referencing that fact.

How the Watchman and Citation Officer Plug In

The Watchman operates as a scheduled Edge Function that runs every 6 hours, checking a rotating subset of `verified_facts` against their original sources. If a source URL returns a 404, or if the content has materially changed, the Watchman updates the

`still_valid` field in `citation_registry` and creates an alert in the Notification Engine. The Citation Officer is an on-demand agent triggered whenever new content is submitted to the system — it validates sources, assigns confidence scores, and either approves the entry into `verified_facts` or rejects it with an explanation.

What Accounting Systems Hook In

The Source of Truth Database does not directly connect to accounting systems. Financial data flows through the KPI Ticker Feed (Connection 5). However, the Source of Truth does store **financial policies** — things like budget approval thresholds, spending authority levels, and financial reporting requirements that govern how the KPI data is interpreted.

Recommended Database Technology

PostgreSQL via Supabase is the recommended technology. The Source of Truth requires strong ACID compliance (facts must be consistent), full-text search (for finding relevant facts quickly), JSONB columns (for flexible metadata storage), and Row-Level Security (to control which buildings can see which facts). PostgreSQL provides all of these natively, and Supabase adds the real-time subscription layer, auto-generated REST API, and Edge Functions for the Watchman and Citation Officer logic.

Schema Sketch

```
CREATE TABLE verified_facts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    building_id UUID REFERENCES buildings(id),
    category TEXT NOT NULL, -- 'policy', 'research', 'financial',
'operational'
    statement TEXT NOT NULL,
    source_url TEXT,
    citation_id UUID REFERENCES citation_registry(id),
    verified_by TEXT NOT NULL, -- 'watchman', 'citation_officer', 'human'
    verified_at TIMESTAMPTZ DEFAULT now(),
    confidence_score DECIMAL(3,2) CHECK (confidence_score BETWEEN 0 AND 1),
    is_active BOOLEAN DEFAULT true,
    metadata JSONB DEFAULT '{}'
);

-- RLS Policy: Users can only see facts for their building + global facts
ALTER TABLE verified_facts ENABLE ROW LEVEL SECURITY;
CREATE POLICY "building_isolation" ON verified_facts
    USING (building_id = current_setting('app.current_building')::UUID
           OR building_id IS NULL);
```

Build Complexity

Medium. The schema is straightforward, but the Watchman's continuous validation logic and the Citation Officer's source-checking pipeline require careful implementation. The main challenge is defining "what counts as a valid source" and building the automated checking pipeline.

---

4. Connection 2: Local North Star Database

What It Is

Each Bingo Building has its own Local North Star — a specific, measurable goal that defines success for that initiative. The Local North Star is derived from the Global North Star (above the HB1000 line) but is tailored to the building's specific context. For example, if the Global North Star is "Make it less expensive to be poor," the Cloud

Butterfly building's Local North Star might be "Capture and validate 100 community-sourced financial insights per quarter."

What Data It Stores

Table	Purpose	Key Fields	Write Frequency
`local_north_stars`	Per-building goal definition	`id`, `building_id`, `statement`, `metric_target`, `metric_current`, `deadline`, `version`, `approved_by`	Monthly
`north_star_history`	Version history of changes	`id`, `north_star_id`, `previous_statement`, `new_statement`, `changed_by`, `change_reason`, `approved_by_pope`	On change
`alignment_checks`	How well building aligns to global	`id`, `building_id`, `global_north_star_id`, `alignment_score`, `checked_at`, `checked_by`	Weekly

How It Is Stored, Versioned, and Protected from Drift

The Local North Star is stored as a versioned record in PostgreSQL. Every change creates a new version in `north_star_history`, preserving the complete evolution of the goal. The current active version is always the latest approved entry. **Drift protection** is handled by the North Star Keeper agent (center of the rooftop), which runs a weekly alignment check comparing the Local North Star against the Global North Star. If the alignment score drops below 0.7 (on a 0-1 scale), the North Star Keeper flags it to the Pope for review.

Who Can Change It and With What Approval

Only the Pope of the building can change the Local North Star, and the change requires a **two-sign approval**: the Pope must approve, and the change must be countersigned by either the Global North Star owner (Tim) or a designated delegate. This mirrors the Constitutional Memory System's Fireproof Safe protection — the most important things require the most deliberate changes.

How It Connects to the Dashboard

The Local North Star appears at the top of every Building View as a persistent banner. The `metric_current` vs `metric_target` comparison drives a progress indicator. The alignment score appears as a small badge — green (above 0.8), amber (0.7-0.8), or red (below 0.7). The Pope Console includes a “Review North Star” action button that opens the full history and alignment analysis.

Build Complexity

Simple. This is a well-defined CRUD operation with versioning. The main complexity is in the alignment-checking logic, which requires comparing semantic meaning (not just text) between the Local and Global North Stars. This could use an LLM call via Edge Function to compute semantic similarity.

---

5. Connection 3: Bingo Card State Engine

What It Is

The Bingo Card State Engine is the heart of the accountability system. Each Pope has one Bingo Card — a 5x5 grid (B-I-N-G-O columns) where each square represents a cluster of sub-initiatives that must ALL be completed before the square lights up. The center square is always PTK (Promises to Keep) — the Pope’s personal commitment anchor, always lit.

What Data It Stores

Table	Purpose	Key Fields	Write Frequency
`bingo_cards`	One per building	`id`, `building_id`, `pope_id`, `created_at`, `status`	On creation
`bingo_squares`	25 per card	`id`, `card_id`, `position` (B1-O5), `label`, `is_lit`, `is_ptk`, `lit_at`, `verified_by`	On completion
`sub_initiatives`	N per square	`id`, `square_id`, `title`, `description`, `assigned_to`, `status`, `completed_at`, `verified_at`	Daily
`verification_chain`	8-step audit per square	`id`, `square_id`, `step_number`, `step_name`, `passed`, `agent_id`, `checked_at`, `notes`	On verification
`bingo_events`	Celebration + history	`id`, `card_id`, `event_type` (square_lit, bingo, blackout), `details`, `occurred_at`	On event

The PEARL Diamond Column Mapping

The five columns of the Bingo Card map to the PEARL Diamond framework, creating a strategic structure behind what appears to be a simple game board.

Column	Letter	PEARL Dimension	Focus Area
1	B	**P**rogress	Revenue, growth, forward momentum
2	I	**E**ngagement	Team health, participation, morale
3	N	**A**lignment	North Star consistency, values adherence
4	G	**R**esilience	Risk mitigation, contingency, hardening
5	O	**L**egacy	Long-term impact, Ruby Red outcomes

How Squares Light Up: The 8-Step Verification Chain

A square does not light up simply because someone clicks a button. It goes through an 8-step verification chain, each step handled by a different agent or process.

Step	Agent/Process	What It Checks	Can Block?
1	Human Worker	Declares all sub-initiatives complete	Yes
2	Lion (Verifier)	Validates each sub-initiative has evidence	Yes
3	Owl (Source of Truth)	Checks claims against Source of Truth DB	Yes
4	Citation Officer	Audits sources and citations	Yes
5	North Star Keeper	Confirms alignment with Local North Star	Advisory
6	Golden Retriever	Reviews for Ruby Red language/empathy	Advisory
7	System	Marks all sub-initiatives as verified	Automatic
8	System	Evaluates square (all subs done = lit)	Automatic

Steps 1-4 are **blocking** — if any fails, the square cannot light up until the issue is resolved. Steps 5-6 are **advisory** — they flag concerns but do not prevent completion. Steps 7-8 are **automatic** — they execute the state change once the blocking steps pass.

The PTK (Promises to Keep) Free Space

The center square (N3 position) is always PTK — the Pope's personal commitment anchor. It is always lit from the moment the card is created. It represents the promises the Pope has already made and is personally accountable for. Clicking on PTK opens a special panel showing the Pope's personal commitments, their status, and any notes. PTK cannot be un-lit. It is a permanent reminder that the Pope's word is their bond.

BINGO Detection

The system checks for BINGO (5 in a row — horizontal, vertical, or diagonal) after every square lights up. There are 12 possible BINGO patterns on a 5x5 card. When BINGO is achieved, a celebration event fires: the building glows brighter in the City View, a notification goes to all team members, and the event is logged in `bingo_events`. A **Blackout** (all 25 squares lit) is the ultimate achievement — the initiative is fully complete.

Build Complexity

High. The Bingo Card State Engine is the most complex component in the system. The 8-step verification chain requires orchestrating multiple AI agents in sequence, handling blocking vs. advisory steps, managing timeouts and retries, and maintaining a complete audit trail. The state machine logic (unlit → pending → verified → lit) must be bulletproof — a square that lights up incorrectly undermines the entire accountability model.

---

6. Connection 4: AI Agent Registry & Orchestration

What It Is

The AI Agent Registry is the database and orchestration layer that manages all AI agents in the Rooftop Society. Each building has its own roster of 9 core agents (Inner Ring + Outer Ring), plus the Voice of Concern, Angel of Your Better Nature, and visiting Wisdom Giants. The registry tracks each agent's current state, task queue, autonomy level, and activity history.

What Data It Stores

Table	Purpose	Key Fields	Write Frequency
`agents`	Agent definitions	`id`, `building_id`, `name`, `role`, `animal_metaphor`, `ring_position`, `autonomy_level`, `is_active`	On setup
`agent_tasks`	Task queue per agent	`id`, `agent_id`, `task_type`, `priority`, `payload`, `status`, `assigned_at`, `completed_at`	Continuous
`agent_activity_log`	Everything an agent does	`id`, `agent_id`, `action_type`, `description`, `input_data`, `output_data`, `tokens_used`, `cost`, `timestamp`	Continuous
`autonomy_scores`	Trust level per agent	`id`, `agent_id`, `score` (1-5), `last_evaluated`, `evaluation_notes`, `adjusted_by`	Weekly
`agent_escalations`	When agents escalate to humans	`id`, `agent_id`, `escalation_level`, `reason`, `target_human`, `resolved`, `resolved_at`	On escalation

The 9 Core Agent Positions

Position	Ring	Name	Animal	Primary Function	LLM Model
1	Inner	Lion	Lion	Project verification, quality gate	Claude 3.5 Sonnet
2	Inner	Hawk	Hawk	Situation monitoring, early warning	GPT-4o
3	Inner	Retriever	Golden Retriever	Ruby Red empathy check, language review	Claude 3.5 Sonnet
4	Inner	Owl	Owl	Source of Truth queries, research	Gemini 1.5 Pro
5	Outer	Bear	Bear	“I Got a Guy” connector, resource finder	GPT-4o
6	Outer	Herald	Peacock	Marketing coordination, communications	Claude 3.5 Sonnet
7	Outer	Researcher	Fox	Deep research, competitive analysis	Gemini 1.5 Pro
8	Outer	Scheduler	Beaver	Calendar, deadlines, ritual management	GPT-4o-mini
9	Center	North Star Keeper	Eagle	Alignment checking, drift detection	Claude 3.5 Sonnet

The 5-Level Autonomy Score

Each agent operates at one of five autonomy levels, which determines how much they can do without human approval.

Level	Name	What the Agent Can Do	Approval Required
1	Manual	Execute only when explicitly told	Every action
2	Assisted	Suggest actions, human approves	Before execution
3	Supervised	Execute routine tasks, flag exceptions	Exceptions only
4	Autonomous	Execute within defined boundaries	Boundary violations
5	Trusted	Full autonomy within constitutional limits	Constitutional violations

Autonomy is **earned, not granted**. A new agent starts at Level 1 and can only advance through demonstrated competence over time. The Pope can revoke autonomy instantly — trust is earned slowly and lost quickly. The Learning Loop V13 cycle evaluates agent performance and recommends autonomy adjustments.

Orchestration Pattern

Agent orchestration follows a **task queue pattern** with priority scheduling. When an event occurs (e.g., a sub-initiative is marked complete), the system creates tasks in the appropriate agent queues. The Lion gets a “verify completion” task, the Owl gets a “check sources” task, and so on. Each agent processes its queue independently, writing results back to the database. The verification chain coordinator (an Edge Function) monitors all agent responses and advances the chain when all blocking steps are complete.

For LLM calls, each agent has a configured model and system prompt stored in the `agents` table. The Edge Function constructs the prompt with relevant context from the database, calls the appropriate LLM API, parses the response, and stores the result in `agent_activity_log`. Token usage and cost are tracked per agent per building, enabling cost allocation and budget management.

Build Complexity

High. This is the second most complex component after the Bingo Card State Engine. The challenges include: managing concurrent agent execution without race conditions, handling LLM API failures gracefully (retries, fallbacks), implementing the autonomy score evaluation logic, and building the escalation ladder that respects human authority while enabling AI efficiency.

---

7. Connection 5: KPI Ticker Tape Feed

What It Is

The KPI Ticker Tape is a scrolling banner that runs around the top of each Bingo Building, displaying real-time financial and operational metrics. It is the Pope's at-a-glance health check — without opening any dashboard, the Pope can see whether the numbers are moving in the right direction just by looking at the building.

What Data It Stores

Table	Purpose	Key Fields	Write Frequency
`kpi_definitions`	What KPIs each building tracks	`id`, `building_id`, `name`, `unit`, `target`, `direction` (up/down), `source_system`	On setup
`kpi_readings`	Time-series metric values	`id`, `kpi_id`, `value`, `recorded_at`, `source`	Every 15 min
`kpi_alerts`	Threshold breach notifications	`id`, `kpi_id`, `alert_type`, `threshold`, `actual_value`, `triggered_at`	On breach
`kpi_display_config`	Ticker tape formatting	`id`, `building_id`, `scroll_speed`, `display_order`, `color_rules`	On config

What Data Feeds the Ticker

The ticker tape pulls from multiple sources, each with different update frequencies.

Source	Data Type	Update Frequency	Integration Method
QuickBooks/Xero	Revenue, expenses, cash flow	Every 15 minutes	REST API polling via Edge Function
Bingo Card Engine	Squares lit, completion %	Real-time	Database trigger
Agent Registry	Active agents, tasks completed	Real-time	Database trigger
Learning Loop	Current score, loop count	On loop completion	Database trigger
CRM	Leads, conversions, pipeline	Every 30 minutes	REST API polling via Edge Function
GitHub	Commits, PRs, deployments	Webhook	GitHub webhook → Edge Function

How It Connects to the Dashboard

The ticker tape is rendered as a CSS-animated scrolling banner in the Building View. It subscribes to the `kpi_readings` table via Supabase Realtime — when a new reading is inserted, the ticker updates instantly without page refresh. Color coding follows a simple rule: green if the value is moving toward the target, amber if flat, red if moving away. The Pope can click any KPI in the ticker to drill down into the full time-series chart.

Build Complexity

Medium. The time-series storage is straightforward (PostgreSQL with TimescaleDB extension if needed for scale). The main complexity is in building reliable connectors to external accounting systems (QuickBooks and Xero have different API patterns, rate limits, and authentication flows) and handling the real-time display updates smoothly.

---

8. Connection 6: Learning Loop V13 Engine

What It Is

The Learning Loop V13 Engine is the continuous improvement system that runs at three levels: task-level (individual sub-initiatives), building-level (entire Bingo Card), and system-level (Bingo City as a whole). It implements the 9-phase V13 protocol, tracking before/after scores, generating mutation logs, and feeding results back into the Autonomy Score Engine.

What Data It Stores

Table	Purpose	Key Fields	Write Frequency
`learning_loops`	Each loop execution	`id`, `building_id`, `level` (task/building/system), `trigger`, `score_before`, `score_after`, `delta`, `status`, `started_at`, `completed_at`	On trigger
`loop_phases`	9 phases per loop	`id`, `loop_id`, `phase_number` (0-8), `phase_name`, `status`, `findings`, `started_at`, `completed_at`	During loop
`mutations`	What changed as a result	`id`, `loop_id`, `mutation_type`, `description`, `target_entity`, `before_state`, `after_state`, `approved_by`	On mutation
`loop_scores`	Historical score tracking	`id`, `building_id`, `score`, `scored_at`, `scored_by`, `methodology`	On completion

The 9 Phases of V13

Phase	Name	What Happens	Agent Responsible
0	Intake	Trigger event captured, context gathered	System
1	Record	Current state documented, baseline established	Lion
2	Innovation	“What could we do better?” research	Researcher
3	Adversarial	Red team challenge, stress test	Hawk
4	Communication	Findings translated for all stakeholders	Herald
5	Evolution	Mutations proposed and evaluated	Owl + Lion
6	Certification	Changes verified against constitution	Citation Officer
7	Drift Agent	Monitor for regression over next cycle	North Star Keeper
8	The Genie	Final score, celebration or correction	System

How It Plugs Into the Accountability Engine

The Learning Loop is triggered by three types of events: (1) **Milestone triggers** — when a Bingo square lights up, a building-level loop runs to evaluate the quality of the completion; (2) **Schedule triggers** — monthly building-level loops and quarterly system-level loops run on a fixed calendar; (3) **Escalation triggers** — when the Hawk detects a significant problem, an emergency loop runs to diagnose and fix it.

The loop’s output directly affects the Autonomy Score. If a loop reveals that an AI agent made errors during verification (Phase 3 catches it), the agent’s autonomy score is reduced. If a loop shows consistent high-quality work, autonomy scores increase. This creates a **feedback loop within the feedback loop** — the system gets better at getting better.

The J-Curve Pattern

Tim has identified that honest accountability follows a J-Curve: scores often drop before they rise. The first Learning Loop on any initiative typically reveals problems that were previously hidden, causing the score to drop. Subsequent loops fix those problems, and

the score rises above the original baseline. The system must display this J-Curve pattern prominently — it is the signature of honest accountability, not a sign of failure.

Build Complexity

High. The Learning Loop Engine requires orchestrating multiple AI agents through a 9-phase pipeline, managing state transitions, handling failures and retries at each phase, computing meaningful scores, and generating human-readable mutation logs. The scoring methodology itself is complex — it must be consistent enough to compare across loops but flexible enough to handle different types of initiatives.

---

9. Connection 7: Cloud Butterfly & Wisdom Giant Capture System

What It Is

Cloud Butterflies and Wisdom Giants are **floating insight resources** — they are not squares on the Bingo Card but rather ambient intelligence that orbits around each building. Cloud Butterflies represent intuitive insights, “what if” moments, and serendipitous connections. Wisdom Giants represent deep expertise from subject matter experts, historical precedents, and best practices. Both are captured, stored, and made available through slide-in panels when clicked.

What Data It Stores

Table	Purpose	Key Fields	Write Frequency
`cloud_butterflies`	Insight captures	`id`, `building_id`, `captured_by`, `insight_text`, `category`, `related_squares`, `confidence`, `captured_at`, `status`	On capture
`wisdom_giants`	Expert knowledge entries	`id`, `building_id`, `expert_name`, `domain`, `insight_text`, `source`, `relevance_score`, `captured_at`	On capture
`insight_connections`	Links between insights and squares	`id`, `insight_id`, `insight_type` (butterfly/giant), `square_id`, `connection_strength`, `connected_by`	On connection
`butterfly_swarms`	Clusters of related insights	`id`, `building_id`, `theme`, `butterfly_ids`, `created_at`, `created_by`	Weekly

How They Appear in the 3D View

In the 3D visualization, Cloud Butterflies are rendered as small, glowing, semi-transparent butterfly shapes that drift and flutter around the building. Their color indicates category (blue for financial, green for operational, amber for strategic, purple for creative). Wisdom Giants appear as tall, glowing silhouettes near the roofline — stationary, authoritative presences. Both are always visible and orbiting.

Clicking on a Cloud Butterfly opens a slide-in panel from the right side showing the insight text, who captured it, when, related Bingo squares, and an option to “connect” it to a specific square or initiative. Clicking on a Wisdom Giant opens a similar panel showing the expert’s name, domain, the insight, and its source.

Capture Mechanism

Cloud Butterflies are captured through multiple channels: (1) a “Capture Butterfly” button in the dashboard that opens a quick-entry form; (2) voice capture via the Retriever

agent (the Pope speaks an insight, the Retriever transcribes and stores it); (3) automated capture when the Researcher agent discovers something relevant during research; (4) import from external sources (email, Slack messages flagged as insights).

Wisdom Giants are captured through: (1) manual entry by team members who consult with external experts; (2) automated extraction from research papers, articles, and books processed by the Owl; (3) import from the Source of Truth Database when a verified fact has particular strategic relevance.

Build Complexity

Medium. The data model is straightforward. The main complexity is in the 3D visualization (rendering butterflies and giants as ambient elements) and in the automated capture pipeline (NLP to categorize insights, relevance scoring to connect them to squares).

---

10. Connection 8: Notification & Escalation Engine

What It Is

The Notification & Escalation Engine is the accountability enforcement layer. It ensures that the Pope, human team, and AI agents are all informed of what needs attention, what is overdue, and what requires immediate action. It implements the 5-rung escalation ladder: Nudge, Flag, Brief, Challenge, and Dead Man’s Switch.

What Data It Stores

Table	Purpose	Key Fields	Write Frequency
`notifications`	All notifications	`id`, `building_id`, `recipient_id`, `type`, `channel`, `subject`, `body`, `priority`, `sent_at`, `read_at`	Continuous
`escalation_rules`	When to escalate	`id`, `building_id`, `trigger_condition`, `escalation_level`, `time_threshold`, `target_role`	On config
`escalation_events`	Active escalations	`id`, `notification_id`, `current_level`, `started_at`, `last_escalated_at`, `resolved_at`, `resolved_by`	On escalation
`notification_preferences`	Per-user channel prefs	`id`, `user_id`, `channel` (email/slack/push/sms), `enabled`, `quiet_hours_start`, `quiet_hours_end`	On config

The 5-Rung Escalation Ladder

Rung	Name	Trigger	Action	Recipient
1	Nudge	Task 1 day overdue	Gentle in-app reminder	Assigned worker
2	Flag	Task 3 days overdue	Dashboard flag + email	Worker + team lead
3	Brief	Task 7 days overdue	Formal briefing document	Pope
4	Challenge	Task 14 days overdue	Hawk presents risk analysis	Pope + full team
5	Dead Man's Switch	Task 30 days overdue OR Pope unresponsive for 7 days	System freezes the building	Everyone

The Dead Man's Switch is the ultimate accountability mechanism. It does not take over — it **stops everything**. If the Pope has been unresponsive for 7 days, or if a critical task has been overdue for 30 days with no action, the system freezes the building. No new tasks can be created, no squares can be verified, no agents can execute. The only way to unfreeze is for the Pope to personally re-engage and acknowledge the situation. This ensures human authority is never overridden — the AI simply refuses to continue without human oversight.

Multi-Channel Delivery

Notifications are delivered through multiple channels based on priority and user preferences. Critical alerts (Rung 3+) always go to email AND push notification regardless of preferences. Routine notifications (Rung 1-2) respect quiet hours and channel preferences. The system tracks delivery confirmation and read receipts to ensure important notifications are actually seen.

Build Complexity

Medium. The notification system itself is a well-understood pattern. The complexity is in the escalation logic (managing timers, handling edge cases like weekends and holidays, ensuring the Dead Man's Switch doesn't trigger falsely) and in the multi-channel delivery (integrating with email services, Slack API, push notification services, and SMS gateways).

---

11. Connection 9: Accounting & Financial Integration

What It Is

The Accounting & Financial Integration connects Bingo City to the real-world financial systems that each initiative uses. This is what makes the KPI Ticker Tape show real numbers instead of placeholders. Each building may use a different accounting system (some use QuickBooks, some use Xero, some use spreadsheets), so the integration layer must be flexible.

What Data It Stores

Table	Purpose	Key Fields	Write Frequency
`financial_connections`	Per-building accounting setup	`id`, `building_id`, `provider` (quickbooks/xero/manual), `credentials_encrypted`, `last_sync`, `sync_status`	On config
`financial_snapshots`	Point-in-time financial data	`id`, `building_id`, `snapshot_date`, `revenue`, `expenses`, `cash_flow`, `accounts_receivable`, `accounts_payable`, `raw_data`	Every 15 min
`budget_allocations`	Budget per building	`id`, `building_id`, `category`, `allocated_amount`, `spent_amount`, `remaining`, `period`	Monthly

Integration Patterns

Provider	Auth Method	Data Access	Sync Frequency	Complexity
QuickBooks Online	OAuth 2.0	REST API v3	Every 15 min	Medium
Xero	OAuth 2.0	REST API	Every 15 min	Medium
Manual Entry	N/A	Dashboard form	On entry	Simple
CSV Import	N/A	File upload	On upload	Simple

For QuickBooks and Xero, the integration uses OAuth 2.0 for authentication (the Pope authorizes the connection once, and the system stores encrypted refresh tokens). An Edge Function runs every 15 minutes, pulling the latest financial data via REST API, transforming it into the standardized `financial_snapshots` format, and inserting it into the database. The Supabase Realtime layer then pushes the update to any connected dashboard.

For buildings that do not use formal accounting software (common for smaller initiatives like a farmers market stall), the system provides a simple manual entry form and CSV import option. The Pope or a team member can enter key financial figures directly.

Security Considerations

Financial data is the most sensitive data in the system. All credentials are encrypted at rest using Supabase Vault. Financial data is isolated by `building_id` with RLS policies — no building can see another building's financial data. API calls to accounting providers are logged in the audit trail. The Pope must explicitly authorize each financial connection, and connections can be revoked at any time.

Build Complexity

Medium. The QuickBooks and Xero APIs are well-documented but have quirks (rate limiting, webhook reliability, token refresh edge cases). The main challenge is handling the variety of accounting setups across 30-40 buildings — some will use QuickBooks, some Xero, some nothing at all. The system must gracefully handle all scenarios.

---

12. Connection 10: Authentication, Authorization & Audit Trail

What It Is

The Auth, Authorization, and Audit Trail system controls who can access what, enforces role-based permissions, and maintains a complete record of every action taken in the system. This is the Constitutional Memory System in digital form — the Fireproof Safe (immutable audit log), the Village Crier (role-based access broadcasting), the Checkpoint (verification gates), and the Watchtower (continuous monitoring).

What Data It Stores

Table	Purpose	Key Fields	Write Frequency
`users`	All human users	`id`, `email`, `name`, `role`, `building_ids`, `created_at`, `last_login`	On registration
`roles`	Role definitions	`id`, `name` (pope/team_lead/worker/observer), `permissions`, `building_scope`	On config
`audit_log`	Every action ever taken	`id`, `user_id`, `agent_id`, `action`, `entity_type`, `entity_id`, `before_state`, `after_state`, `ip_address`, `timestamp`	Continuous
`constitutional_directives`	The 23 active directives	`id`, `directive_text`, `category`, `priority`, `created_at`, `approved_by`, `is_active`	Rarely
`sessions`	Active user sessions	`id`, `user_id`, `token`, `device_info`, `created_at`, `expires_at`	On login

Role-Based Access Control (RBAC)

Role	Scope	Can View	Can Edit	Can Approve	Can Delete
Pope	Own building	Everything	Everything	Everything	With 2-sign
Team Lead	Own building	Everything	Assigned areas	Sub-initiatives	No
Worker	Own building	Assigned tasks	Own tasks	No	No
Observer	Own building	Dashboard only	No	No	No
System Admin	All buildings	Everything	System config	System changes	With 2-sign
AI Agent	Own building	Per autonomy level	Per autonomy level	Per autonomy level	No

The Audit Trail as Constitutional Memory

The audit log is **append-only** — entries can never be modified or deleted. This is the digital equivalent of the Fireproof Safe. Every action in the system generates an audit entry: every login, every task completion, every agent action, every North Star change, every escalation, every square verification. The `before_state` and `after_state` fields capture the full diff of what changed, enabling complete reconstruction of the system's history at any point in time.

The Watchtower agent monitors the audit log in real-time, looking for anomalies: unusual login patterns, rapid state changes that might indicate automation errors, actions taken outside normal business hours, and any attempts to circumvent the verification chain. When the Watchtower detects an anomaly, it creates an alert in the Notification Engine.

Build Complexity

Medium. Supabase provides built-in authentication (GoTrue) and Row-Level Security, which handles 80% of the auth/authz requirements out of the box. The main complexity is in building the comprehensive audit trail (capturing before/after states for every action

without impacting performance) and implementing the Watchtower's anomaly detection logic.

---

13. Recommended Tech Stack

Based on the analysis of all 10 connection areas, the following tech stack is recommended for Bingo City.

Layer	Technology	Why This Choice
Database	PostgreSQL 16 (via Supabase)	ACID compliance, RLS for multi-tenancy, JSONB for flexibility, full-text search, mature ecosystem
Real-Time	Supabase Realtime	WebSocket subscriptions on database changes, broadcast channels, presence tracking — all built-in
Auth	Supabase Auth (GoTrue)	JWT-based, social logins, magic links, integrates directly with RLS policies
API	Supabase Auto-Generated REST + GraphQL	Every table gets an API automatically, no backend code needed for CRUD
Serverless Logic	Supabase Edge Functions (Deno)	TypeScript, runs at the edge, handles webhooks, AI agent orchestration, accounting API calls
File Storage	Supabase Storage (S3-compatible)	Documents, images, Cloud Butterfly attachments, audit exports
Frontend	React 19 + Tailwind CSS 4 + shadcn/ui	Already scaffolded in the current project, proven stack
3D Visualization	Three.js + React Three Fiber	Already installed, handles the Bingo City 3D view
Charts	Recharts	Already installed, handles KPI time-series charts
AI/LLM	Multi-provider (OpenAI, Anthropic, Google)	Different agents use different models based on their task requirements
Accounting	QuickBooks API v3 + Xero API	Direct REST integration via Edge Functions
Notifications	Resend (email) + Slack API + Web Push	Multi-channel delivery with preference management
Monitoring	Supabase Dashboard + custom Watchtower	Built-in metrics plus custom anomaly detection

Why Supabase Over Alternatives

Alternative	Why Not
Firebase	Firestore's document model is awkward for relational data (Bingo Cards have deep relational structure). No built-in RLS equivalent.
Airtable	Great for prototyping but lacks real-time subscriptions, RLS, and the performance needed for 30-40 concurrent buildings.
Notion	Not a database — it is a document tool. Cannot handle the transactional requirements of the Bingo Card State Engine.
Custom PostgreSQL + Express	Requires building everything Supabase provides for free (auth, real-time, auto-API, storage). Months of extra work.
Hasura	Good GraphQL layer but less integrated than Supabase (no built-in auth, storage, edge functions). More ops overhead.

Supabase provides the best combination of **developer experience** (auto-generated APIs, built-in auth), **real-time capability** (WebSocket subscriptions on any table), **multi-tenancy support** (RLS policies), and **serverless compute** (Edge Functions for business logic). It is also **open source** and **self-hostable**, which aligns with the HB1000 principle of portability and long-term sustainability.

---

14. Multi-Tenant Architecture Design

The Pattern: Shared Database with Row-Level Security

For 30-40 Bingo Buildings, the recommended multi-tenancy pattern is a **shared database with tenant discriminator columns and Row-Level Security (RLS)** [1]. This means all buildings share the same PostgreSQL database and the same tables, but every table has a `building_id` column, and RLS policies ensure that queries only return data for the current user's building.

This approach was chosen over three alternatives based on the research.

Approach	Scale Sweet Spot	Why Not for Bingo City
Database per building	10s of tenants	30-40 separate databases is manageable but creates migration nightmares and prevents cross-building analytics
Schema per building	100s of tenants	Adds migration complexity across 30-40 schemas, no real benefit over RLS at this scale
Shared + RLS	**Millions of tenants**	**Perfect fit — single schema, single migration path, cross-building queries possible, RLS enforces isolation**
Citus extension	Millions of tenants	Overkill for 30-40 buildings, adds operational complexity

RLS Implementation

Every table in the system includes a `building_id` column. When a user authenticates, their JWT token includes their `building_id` claim. RLS policies use this claim to filter all queries automatically.

```
-- Example: Set the current building from the JWT
CREATE OR REPLACE FUNCTION current_building_id()
RETURNS UUID AS $$
    SELECT (current_setting('request.jwt.claims', true)::json->>'building_id')::UUID;
$$ LANGUAGE sql STABLE;

-- Example: RLS policy on bingo_squares
ALTER TABLE bingo_squares ENABLE ROW LEVEL SECURITY;
CREATE POLICY "building_isolation" ON bingo_squares
    FOR ALL
    USING (
        card_id IN (
            SELECT id FROM bingo_cards
            WHERE building_id = current_building_id()
        )
    );
```

Cross-Building Analytics

For the City View (all buildings at a glance), the system needs to aggregate data across buildings. This is handled by a **service role** that bypasses RLS — used only by Edge Functions that compute city-wide metrics. The service role is never exposed to the frontend. City-wide aggregates are computed by Edge Functions and stored in a separate `city_metrics` table that all users can read.

---

15. Event-Driven Architecture & Real-Time Sync

The Pattern: Database Change → Event → Action → Broadcast

The following diagram shows the four primary event-driven data flows in Bingo City.

Every significant action in Bingo City follows the same pattern: a database change triggers an event, the event is processed by an Edge Function, the Edge Function takes action (calling an AI agent, sending a notification, updating a metric), and the result is broadcast to all connected dashboards via WebSocket.

Flow 1: Square Lights Up — A human worker marks a task complete → the Lion agent verifies → the Owl checks sources → the Citation Officer audits → the Retriever checks

Ruby Red language → if all pass, the square's `is_lit` field is set to `true` → Supabase Realtime broadcasts the change → every connected dashboard sees the square light up instantly.

Flow 2: KPI Ticker Update — An accounting API webhook fires (or a scheduled poll completes) → an Edge Function transforms the data → the new reading is inserted into `kpi_readings` → Supabase Realtime broadcasts → the ticker tape on the building updates in real-time.

Flow 3: Accountability Escalation — The Hawk agent detects an overdue task → the Escalation Engine evaluates severity → a notification is queued → multi-channel delivery (email + Slack + push) → the Pope's dashboard shows a red alert.

Flow 4: Learning Loop V13 Cycle — A trigger event fires (milestone, schedule, or escalation) → the 9-phase engine runs (Intake through Genie) → a score and report are generated → mutations are logged → the Autonomy Score is adjusted → all changes broadcast to connected dashboards.

Supabase Realtime Channels

Each building gets its own Realtime channel: `building:{building_id}`. The dashboard subscribes to this channel on load and receives all updates for that building. For the City View, the dashboard subscribes to a `city:metrics` channel that receives aggregated updates.

```
// Example: Subscribe to building updates
const channel = supabase
    .channel(`building:${buildingId}`)
    .on('postgres_changes',
        { event: '*', schema: 'public', table: 'bingo_squares',
          filter: `card_id=eq.${cardId}` },
        (payload) => {
            // Update the square in the UI
            updateSquare(payload.new);
        }
    )
    .on('postgres_changes',
        { event: 'INSERT', schema: 'public', table: 'kpi_readings',
          filter: `building_id=eq.${buildingId}` },
        (payload) => {
            // Update the ticker tape
            updateTicker(payload.new);
        }
    )
    .subscribe();
```

---

16. Phased Build Roadmap

The following diagram shows the recommended 5-phase build approach.

Phase 1: Foundation (Weeks 1-4)

The first phase establishes the core infrastructure. This includes setting up the Supabase project, implementing authentication and RBAC (Pope, Team, AI roles), creating the core database schema (buildings, cards, squares, sub-initiatives), building the Source of Truth tables, configuring Local North Star per-building settings, and creating the basic dashboard shell with navigation between City View, Building View, and Card View. At the end of Phase 1, the Pope can log in, see their building, view their Bingo Card, and manually manage sub-initiatives. **Effort: Medium.**

Phase 2: Bingo Engine (Weeks 5-8)

The second phase builds the accountability engine. This includes implementing the Bingo Card state machine (square states: unlit, pending, verified, lit), building the square completion logic with the 8-step verification chain, creating the sub-initiative tracking system, implementing the PTK free space mechanic, and enabling real-time square updates via Supabase Realtime. At the end of Phase 2, squares can light up through the full verification chain, and the Pope can see real-time progress. **Effort: High.**

Phase 3: AI Agents (Weeks 9-12)

The third phase brings the Rooftop Society to life. This includes building the Agent Registry and task queue system, integrating LLM APIs for each agent (Lion, Hawk, Retriever, Owl, etc.), implementing the 5-level Autonomy Score engine, building the 5-rung escalation ladder, creating agent activity logging, and building the Rooftop Status dashboard. At the end of Phase 3, AI agents are actively participating in the verification chain, monitoring for issues, and escalating when needed. **Effort: High.**

Phase 4: Integrations (Weeks 13-16)

The fourth phase connects Bingo City to the outside world. This includes building the QuickBooks/Xero accounting connectors, implementing the KPI ticker tape feed, building the Cloud Butterfly capture system, creating the multi-channel notification engine, implementing the Learning Loop V13 engine, and connecting GitHub activity tracking. At the end of Phase 4, the system is fully connected to real-world data sources and the accountability rituals are automated. **Effort: High.**

Phase 5: Polish (Weeks 17-20)

The fifth phase adds the visual and experiential layer. This includes building the 3D city visualization with Three.js, implementing BINGO celebration effects, creating cross-building analytics for the City View, building the Orange Lifeline dashboard, implementing Wisdom Giant slide-in panels, and ensuring mobile responsive views. At the end of Phase 5, the system is production-ready with the full visual experience. **Effort: Medium.**

---

17. Complexity & Effort Estimates

Per-Connection Summary

#	Connection	Database	Build Complexity	Estimated Effort	Phase
1	Source of Truth	PostgreSQL + Supabase	Medium	2 weeks	1
2	Local North Star	PostgreSQL + Supabase	Simple	1 week	1
3	Bingo Card State Engine	PostgreSQL + Supabase	High	4 weeks	2
4	AI Agent Registry	PostgreSQL + Edge Functions	High	4 weeks	3
5	KPI Ticker Tape	PostgreSQL + TimescaleDB	Medium	2 weeks	4
6	Learning Loop V13	PostgreSQL + Edge Functions	High	3 weeks	4
7	Cloud Butterfly / Wisdom Giant	PostgreSQL + Supabase Storage	Medium	2 weeks	4
8	Notification & Escalation	PostgreSQL + Resend + Slack	Medium	2 weeks	4
9	Accounting Integration	Edge Functions + OAuth	Medium	2 weeks	4
10	Auth, RBAC & Audit Trail	Supabase Auth + RLS	Medium	2 weeks	1

Total Effort

Category	Estimate
Total development time	20 weeks (5 months)
Team size	4 people (2 full-stack, 1 AI/ML, 1 DevOps)
Monthly Supabase cost (Pro)	~$25/month base + usage
Monthly LLM API cost	~$200-500/month (depends on agent activity)
Monthly infrastructure	~$50/month (CDN, email, monitoring)

Risk Factors

Risk	Impact	Mitigation
LLM API reliability	Agent tasks fail or timeout	Multi-provider fallback (if OpenAI fails, try Claude)
Accounting API rate limits	KPI ticker goes stale	Cache aggressively, degrade gracefully
WebSocket connection limits	Dashboard stops updating	Supabase handles this at scale, but monitor
Scope creep on AI agents	Phase 3 takes 8 weeks instead of 4	Strict MVP per agent, iterate after launch
Pope adoption resistance	System goes unused	Start with 1 building, prove value, then expand

---

18. References

[1] Craig Kerstiens, “Designing Your Postgres Database for Multi-tenancy,” Crunchy Data Blog, November 2023. https://www.crunchydata.com/blog/designing-your-postgres-database-for-multi-tenancy

[2] Supabase Documentation, “Architecture,” Supabase, 2024. https://supabase.com/docs/guides/getting-started/architecture

[3] EndZone Leadership, “Hyper-Accountability as a Leadership Strategy,” EndZone Leadership, 2024. https://endzoneleadership.com/leadership-traits/hyper-accountability/

[4] Kearney, “Mission Command: Applying a Military Leadership Philosophy to High-Performance Teams,” Kearney, 2023. https://www.kearney.com/service/organization/article/-/insights/mission-command-applying-a-military-leadership-philosophy-to-high-performance-teams

[5] HB1000 Bingo City Architecture Repository, “BINGO_CITY_ARCHITECTURE.md,” GitHub, 2026. https://github.com/timjlatimer/sic-bingo-city-architecture

[6] HB1000 Bingo City Architecture Repository, “Bingo Card Protocol,” GitHub, 2026. https://github.com/timjlatimer/sic-bingo-city-architecture/blob/master/contributions/brain-dump-and-marketing-research/docs/05-BINGO-CARD-PROTOCOL.md

[7] HB1000 Bingo City Architecture Repository, “PEARL Canon,” GitHub, 2026. https://github.com/timjlatimer/sic-bingo-city-architecture/blob/master/contributions/brain-dump-and-marketing-research/docs/09-PEARL-CANON.md

[8] HB1000 Bingo City Architecture Repository, “Learning Loop V13 Workbook,” GitHub, 2026. https://github.com/timjlatimer/sic-bingo-city-architecture/blob/master/contributions/manus-protocol-engineer/learning-loop-v13/learning_loop_v13_workbook.md

---

This report was prepared by the Learning Loop Dashboard Agent as a strategic technical architecture document for the SIC HB1000 Solve Team. It is intended to inform build decisions, not to prescribe implementation details. The recommended architecture should be validated with the full team before development begins.